

HOSTED BY



ELSEVIER

Contents lists available at ScienceDirect

Journal of King Saud University - Computer and Information Sciences

journal homepage: www.sciencedirect.com

Full length article

Pairing-free Proxy Re-Encryption scheme with Equality Test for data security of IoT

Gang Han^{a,b}, Le Li^a, Baodong Qin^{a,*}, Dong Zheng^a^a Xi'an University of Posts and Telecommunications, Xi'an, China^b State Key Laboratory of Integrated Services Networks (Xidian University), Xi'an, China

ARTICLE INFO

MSC:

00-01

99-00

Keywords:

Data recovery

Equality test

Pairing-free

Proxy re-encryption

Secure de-duplication

ABSTRACT

The construction of IoT cloud platform brings great convenience to the storage of massive IoT node data. To ensure security of data collected by the IoT nodes, the data must be encrypted when the nodes upload it. However, it arises some challenge problems, such as efficient retrieval on encrypted data, secure de-duplication and massive data recovery. Proxy Re-Encryption with Equality Test (PREET) can be used to solve these problems. But, existing PREET schemes are based on bilinear pairings, which have low operational efficiency. In order to improve the overall operational efficiency, this paper constructs a Pairing-Free PREET scheme (PF-PREET). Security of this scheme is guaranteed by the Gap Computational Diffie-Hellman assumption in random oracle model. It is demonstrated that the PF-PREET scheme can achieve ciphertext indistinguishability for malicious users and one-wayness for malicious servers. It still has the same security level compared to existing PREET schemes by expanding the scope of the equality test. We discussed the efficiency of the proposed PF-PREET scheme in terms of overall running time, average running time of each phase and time overhead of tag test. The simulated experiment shows a certain improvement compared to PREET schemes using bilinear pairing.

1. Introduction

The emergence of IoT brings huge changes to people's life, such as smart city, smart medical care, smart home, etc, all of which are developing towards wisdom and intelligence. The IoT cloud platform is the mutual integration of IoT and cloud computing technologies and faces many security threats, such as data security (Muhammad et al., 2021). In the development of IoT, data plays an important role and its security is very critical. To protect the security of the data, they first encrypt the data and then transmit the ciphertext to the IoT cloud platform. However, it is difficult to achieve secure de-duplication and data sharing (Thirumalai et al., 2020). These issues may be solved by using the same key to encrypt and decrypt the data. However, such method is not practical in multi-user scenarios. Using Public Key Encryption with Keyword Search (PEKS) (Boneh et al., 2004) and Public Key Encryption with Equality Test (PKEET) (Yang

et al., 2010) to manage data in a multi-device IoT environment is a feasible solution (Thirumalai et al., 2020).

Due to a large number of IoT nodes with a small storage space, users want cloud servers to provide security protection and availability for huge amount of data generated by many IoT nodes. For cloud service providers, they need to provide both secure storage services for users and sufficient storage space for IoT cloud platforms. The ways to enhance the storage space include adding hardware, compressing files, removing repeated files, etc. PKEET, firstly proposed by Yang et al. (2010), is mainly used to solve the problem of secure de-duplication in a multi-user environment. The core idea is that two parties encrypt the data using their public keys and send the ciphertexts to the cloud server. The server uses the equality test algorithm to determine whether the ciphertexts containing the same plaintexts or not, without decrypting the ciphertexts.

* Corresponding author.

E-mail addresses: hangang668866@163.com (G. Han), lilevictory@163.com (L. Li), qinbaodong@xupt.edu.cn (B. Qin), zhengdong_xupt@sina.com (D. Zheng).

Peer review under responsibility of King Saud University.



Production and hosting by Elsevier

<https://doi.org/10.1016/j.jksuci.2024.102105>

Received 12 October 2023; Received in revised form 27 May 2024; Accepted 14 June 2024

Available online 25 June 2024

1319-1578/© 2024 The Authors. Published by Elsevier B.V. on behalf of King Saud University. This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

Since 2010, there have been a large number of PKEET schemes, researching on authorization mode or combining other cryptographic primitives with equality test to build secure and efficient equality test schemes. Consider the following scenario: Alice and Bob upload their ciphertexts to the storage server, and the storage server uses the equality test technique to determine whether the plaintexts sent by the two parties are the same. If the plaintexts are the same, the server only stores the ciphertext sent by Alice, which can save storage space. However, the ciphertext stored in the server is encrypted by Alice's key, Bob cannot decrypt it using his own secret key in traditional PKEET schemes. In order to solve the data recovery problem, this paper combines Proxy Re-Encryption (PRE) and PKEET to build a Pairing-Free Proxy Re-Encryption with Equality Test (PREET) scheme. Alice encrypts the duplicate data and stores it in the cloud. The proxy re-encryption key is generated using Bob's public key and Alice's private key through the proxy re-encryption technique. Then, the proxy server uses the proxy re-encryption key to re-encrypt the original ciphertext under Bob's public key. When Bob receives the transformed ciphertext, he can use his private key to decrypt it, avoiding the leakage of Alice's private key.

A similar cryptographic primitive is the PEKS, proposed by Boneh et al. (2004). It supports keyword retrieval on encrypted data files and is generally applicable to the scenario where the same public key is used for encryption. PEKS mechanism compares whether the keywords extracted from the ciphertext are the same, and there is a certain degree of ambiguity. While the equality test technique can compare the two ciphertexts within the authority range, determining whether the plaintexts corresponding to the ciphertexts of two parties are the same or not.

1.1. Our contribution

As we previously discussed, the IoT data volume is generally very large and there are massive nodes in IoT. Each IoT node has constrained computing and storage resources. In addition, the data originally encrypted by one IoT node, may be shared by the other IoT node. Using traditional PRE scheme to transform the ciphertext is unrealistic as it requires the node of the data owner being online and communicating with each other to generate the re-encryption key. So, it is challenge to realize encrypted data sharing between different nodes while supporting secure de-duplication.

To solve above issues, by combining PRE with PKEET, this paper constructs a new equality test scheme PF-PKEET without bilinear pairing group. It can realize secure data de-duplicate and data sharing, and at the same time can save a large amount of storage space for cloud service providers. Specially, ciphertext transformation keys are generated and managed by a trusted key generation center. Consequently, to sharing encrypted duplication data, IoT nodes do not need to communicate with each other. After experimental simulation analysis, it can effectively reduce the computational overhead of equality test, which is more in line with the application scenario of massive nodes in IoT. The use of PRE encryption primitive, which supports data sharing and data recovery, can effectively reduce unnecessary communication overhead and storage overhead. The main contributions of the scheme are summarized as follows.

- It combines PKEET and PRE, and inherits their advantages. When the data owner and data user perform equality test, it is decided that the plaintexts corresponding to the ciphertexts of both parties are the same or not. If so, the cloud server only saves the ciphertexts of the data owner to save storage space. The data user can use the proxy re-encryption to recover their data completely, forming a complete data security de-duplication closed loop, which is in line with the practical application scenario.
- In the random oracle model, security analysis based on the Gap-CDH problem, for the complicity between type I adversary ma-

licious users proves that our scheme satisfies indistinguishability under selective identity and adaptive selective ciphertext attacks, referred to as IND-sID-CCA security. The attack behavior for type II adversary malicious server proves that this scheme satisfies one-wayness under selective identity and adaptive selective ciphertext attacks, referred to as OW-sID-CCA security.

- Through simulation experiments, the results show that, compared with the existing PREET schemes, the pairing-free proxy re-encryption with equality test scheme constructed by this paper has better performance in terms of computation overhead. The efficiency of our scheme is 20% to 50% higher than the currently known PREET schemes.

1.2. Related work

In Proxy Re-Encryption (PRE), the data owner uses its own public key to encrypt message, and then generates the transformed ciphertext through the proxy re-server. The proxy re-encryption key is generated using the data owner's private key and the data user's public key, the data user can use its private key to decrypt the transformed ciphertext after receiving it, thus protecting the data owner's private key from being leaked. By PRE mechanism, the flexibility of data sharing access mechanism can be improved (Myers and Shull, 2018; Zuo et al., 2018) to reduce the computation overhead and meet the practical scenario requirements. However, many of the PRE schemes are implemented based on the nature of bilinear pairing, so they may be lacking in terms of computational efficiency. To improve the efficiency of PRE, Lu and Li (2016) designed a pairing-free proxy re-encryption scheme for devices with limited computational processing power. Braeken et al. (2017) proposed a pairing-free identity based signcryption scheme with a proxy re-encryption feature. Prasad and Purushothama (2021) proposed an efficient CCA-secure proxy re-encryption scheme without bilinear pairing, which maintains some advantages, such as key optimal, collusion safe and proxy visible. Though these PRE schemes are pairing free, they did not consider functionality of equality test. In addition, there are some PRE schemes proposed for specific application scenarios, such as IoT and cloud computing (Hong and Sun, 2018; Kim and Lee, 2018; Agyekum et al., 2022).

PKEET is a technique to achieve secure data de-duplication. Currently, most schemes are constructed using bilinear pairing to perform equality test. Yang et al. (2010) firstly proposed the concept of equality test, which supports encrypting plaintext messages using different users' public keys and determining whether the plaintext messages are identical or not in the ciphertexts without decrypting them. Since no authorization mechanism is designed, anyone can perform equality test, which is vulnerable to brute force attacks, and difficult to extend to a large number of user scenarios. The papers (Lee et al., 2016; Ma et al., 2015b; Tang, 2012; Huang et al., 2015; Ma, 2016; Ma et al., 2015a; Li et al., 2019) proposed multiple schemes to support a single authorization mode to increase the security of equality test. In the actual IoT environment, different encryption algorithms are used by different IoT nodes, and the equality test of heterogeneous IoT systems has attracted the interest of scholars, and the papers (Hassan et al., 2020; Elhabob et al., 2019, 2020) achieve secure and efficient equality test under heterogeneous systems. Susilo et al. (2022) proposed PKE-MET, which supports equality test among multiple users and multiple ciphertexts, and can effectively reduce the leakage problem of user information. Lin et al. (2022) proposed PKEET-FDA, which allows users to adaptively authorize multiple testers to perform equality test on their ciphertexts. In recent years, PKEET was extended to other primitives to simplify the management of public keys, such as identity-based encryption with equality test (Ma, 2016; Wu et al., 2017) and certificateless public key encryption with equality test (Qu et al., 2018).

Proxy re-encryption with equality test (PREET) was first proposed by Li et al. (2022). It combined PRE and PKEET to achieve efficient retrieval of ciphertexts under different public keys. Though PREET

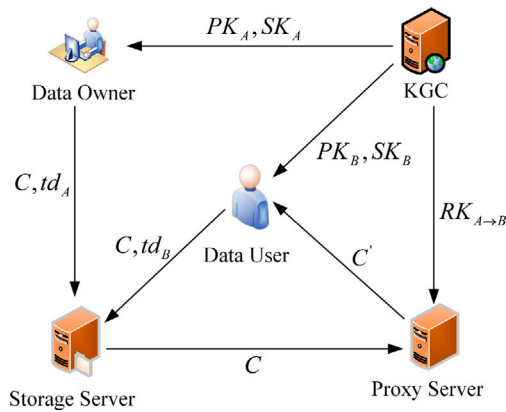


Fig. 1. System model.

can achieve data sharing, it cannot achieve verification of ciphertext integrity when decrypting ciphertexts. Yang et al. (2021) proposed a method to verify integrity ciphertexts and to achieve CCA security for both original and transformed ciphertexts. Chen et al. (2021) proposed BPREET, which inherits the decentralized nature of blockchain.

Among above cryptographic primitives, some ones aim to achieve flexible data sharing or simplify key management in traditional PKI-based infrastructure, such as proxy re-encryption, identity-based encryption and certificateless encryption, and some ones aim to provide secure data de-duplication, such as public key encryption with equality test. Combining data sharing with equality test, especially PREET, is a practical way to solve problems of secure storage and efficient retrieval among multiple-user environments. However, previous PREET schemes are designed over bilinear pairing groups, in which the pairing operations are very expensive.

With the advent of post-quantum cryptography (Kermani and Azarderakhsh, 2016; Canto et al., 2022; Sarker et al., 2022; Elkhatib et al., 2021) and side channel attacks (Canto et al., 2023a; Kaur et al., 2023), recently, some works focus on designing lattice based PRE (Fan and Liu, 2019) and PKEET (Roy et al., 2022), or leakage resilient PRE (Wang, 2018). It is still interesting to design lattice-based PREET or leakage-resilient PREET. Lightweight cryptography (Kermani et al., 2016; Canto et al., 2023c,b) is also an important cryptographic primitive for resource-constrained environment. So, it is necessary to design lightweight PRE or PREET for secure IoT environment (Khashan, 2020).

1.3. Organization

The system model, the definition and the security models of PF-PREET are given in Section 2. The concrete construction and performs correctness analysis are given in Section 3. The security analysis of our scheme is shown in Section 4. In Section 5, we analyze the efficiency of our scheme. Finally, we conclude our work in Section 6.

2. Definition of PF-PREET

2.1. System model

As shown in Fig. 1, the main entities of the scheme are key generation center (KGC), data owner A, data user B, storage server, and proxy server, and the main roles of each entity are as follows.

KGC: The entity generates key pairs of data owners, key pairs of data users, and computes proxy re-encryption keys.

Data owner A: The user obtains its own key pair from KGC. In the encryption phase, it encrypts the data with its public key, generates the ciphertext, generates the authorization trapdoor with its private key,

then transfers the ciphertext and authorization trapdoor to the storage server. When the data owner wants to decrypt the ciphertext uploaded to the cloud server by himself, he can use his private key to decrypt it.

Storage server: The server stores the ciphertext and performs the equality test. When the storage server receives the equality test request, it runs **Test** algorithm using the authorization information provided by both parties to determine whether the ciphertexts of both parties are encrypted by the same plaintext. If the plaintexts of both parties are the same, only the ciphertext of the data owner is retained.

Proxy server: The server receives the proxy re-encryption key from KGC and receives the original ciphertext from the storage server that needs to be re-encrypted. The proxy server uses the proxy re-encryption key to re-encrypt the original ciphertext, generates the transformed ciphertext, and sends the transformed ciphertext to the data user. Here it is assumed that the proxy server is also honest and curious.

Data user B: The data user encrypts the data with its public key and generates an authorization trapdoor. The proxy server re-encrypts the original ciphertext to generate the transformed ciphertext and sends it to the data user, who can decrypt the transformed ciphertext with his private key to get the plaintext message.

2.2. Scheme definition

The PF-PREET scheme consists of nine algorithms. Assuming that the two users involved in the algorithm of the system are data owner A and data user B, the following algorithms are executed.

- Setup (k) $\rightarrow (msk, param)$: This algorithm is run by KGC. Input a security parameter k , it generates a master private key msk and a system parameter $param$.
- Extract ($param, ID, msk$) $\rightarrow (PK, SK)$: This algorithm is run by KGC. Input a system parameter $param$, a user identity ID and a master private key msk , it generates the user's public/secret keys (PK, SK).
- Encrypt ($param, m, PK$) $\rightarrow C$: This algorithm is run by a user. Input a system parameter $param$, a plaintext m and a user's public key PK , it generates an original ciphertext C .
- Decrypt ($param, C, SK$) $\rightarrow m/\perp$: This algorithm is run by a user. Input a system parameter $param$, an original ciphertext C and a user's private key SK . It decrypts the ciphertext to a plaintext message m or a special symbol $\perp$ indicating that the ciphertext is not valid.
- Aut (SK) $\rightarrow td$: This algorithm is run by a user. Input the user's private key SK , it generates a trapdoor td for equality test.
- Test (C_A, td_A, C_B, td_B): This algorithm is run by a storage server. Input ciphertexts C_A and C_B , and authorized trapdoors td_A, td_B of the two parties, it performs the equality test. If the test result is True, it means $m_A = m_B$, the storage server only saves C_A . If $m_A \neq m_B$, the storage server saves C_A and C_B respectively.
- ReKeyGen (ID_A, ID_B, msk, PK_B) $\rightarrow RK_{A \rightarrow B}$: This algorithm is run by KGC. When the ciphertext of data user B is duplicated in the storage server, the proxy server generates the proxy re-encryption key $RK_{A \rightarrow B}$ of the corresponding ciphertext C_A for user B . The $RK_{A \rightarrow B}$ is computed using identities ID_A, ID_B of both users, the system master private key msk and the public key PK_B of data user B . The $RK_{A \rightarrow B}$ is kept secret from the storage server and data user B .
- ReEncrypt ($C, RK_{A \rightarrow B}$) $\rightarrow C'$: This algorithm is run by a proxy server. Input an original ciphertext C and a proxy re-encryption key $RK_{A \rightarrow B}$, it generates a transformed ciphertext C' .
- ReDecrypt ($ID_A, ID_B, C', SK_B, PK_A$) $\rightarrow m$: It is run by data user B . Input a transformed ciphertext C' , a public key PK_A of data owner A , and a private key SK_B of data user B , it decrypts C' to obtain plaintext m or symbol $\perp$ indicating that the ciphertext is not legitimate.

2.3. Secure de-duplication process

The overall data secure de-duplication process includes two parts: data de-duplication and data recovery. Each process is described as follows.

Phase 1: Data de-duplication

- **Data owner:** is a user who uploads a file for the first time. As a data owner, the file owned by the data owner does not have a copy in the cloud server, so the ciphertext needs to be uploaded to the storage server as the only copy of the data.
- **Data user:** is a user who uploads a file not for the first time. When a data user uploads an encrypted file, he/she authorizes the storage server to perform an equality test. When the test result output by the **Test** algorithm is 'True', it indicates that a copy of the same file is already stored in the server, and the data user does not need to upload the same file repeatedly. Otherwise, the data user uploads the ciphertext as the data owner.

Phase 2: data recovery

When a data user needs to recover the files owned by itself, proxy re-encryption is used to achieve it. Considering that in the secure de-duplication system, the data user cannot be online in real time, and the generation of the proxy re-encryption key requires the private key of the data owner and the public key of the data user. To avoid this limitation, the proxy re-encryption key is computed by the trusted third party KGC. When the data user needs to get the file, KGC generates the corresponding proxy re-encryption key based on the result of the equality test and sends it to the proxy server, which uses the key to re-encrypt the original ciphertext into the data user's ciphertext and sends the transformed ciphertext to the data user. After receiving the transformed ciphertext, the data user can decrypt and recover the file by using his own private key.

2.4. Security model

We consider the following two types of adversaries. The type I adversary does not query the equality test key of the challenge user. The type II adversary can query the equality test key of the challenge user. In addition, both types of adversaries can obtain the public and private keys of different users, but not the private key of the challenge user, and can perform re-encryption query, decryption query, or trapdoor query, but cannot query the challenge user's equality test key and proxy re-encryption key at the same time. The type I adversary portrays the conspiracy behavior among malicious users and the attack behavior of malicious proxy server. While type II adversary portrays the attack behavior of a malicious storage server. In the following, the corresponding security models are given for each of these two types of adversaries.

Security Model 1. For Type I adversary, the ciphertext indistinguishability under selective identity and adaptive selective ciphertext attacks is defined by means of **Game 1**, referred to as IND-sID-CCA security.

Game 1. Assuming that the identity of the user to be attacked by Type I adversary $\mathcal{A}_1$ is ID^* . Adversary $\mathcal{A}_1$ and challenger $\mathbb{C}$ interact to complete the following game.

Setup. Input a security parameter k , the challenger $\mathbb{C}$ executes the Setup algorithm to generate a master private key msk and a system parameter $param$, and the challenger sends the system parameter $param$ to the adversary $\mathcal{A}_1$. In addition, challenger runs the key generation algorithm **Extract** to generate the public key PK^* and private key SK^* of the challenger user, and sends the public key PK^* to adversary $\mathcal{A}_1$.

Phase 1. The adversary can adaptively execute the following queries.

- **Key queries (ID_i):** When $\mathcal{A}_1$ asks for the key of ID_i , the challenger $\mathbb{C}$ runs **Extract** algorithm to compute the key of ID_i and the challenger sends the private key SK_{ID_i} , public key PK_{ID_i} to adversary $\mathcal{A}_1$.
- **Decryption queries (ID_i, C):** When $\mathcal{A}_1$ asks for the decryption result of ciphertext C under identity ID_i , the challenger $\mathbb{C}$ runs the Decrypt algorithm to decrypt and returns the decryption result to $\mathcal{A}_1$.
- **Trapdoor queries (ID_i):** When $\mathcal{A}_1$ asks for the trapdoor of ID_i , the challenger $\mathbb{C}$ executes $\text{Aut}(SK_{ID_i})$ algorithm to return the trapdoor td_{ID_i} to $\mathcal{A}_1$.
- **Re-encryption queries (ID_A, ID_B):** When $\mathcal{A}_1$ asks the transformed ciphertext C from ID_A to ID_B , the challenger $\mathbb{C}$ runs the ReKeyGen algorithm to generate the re-encryption key $RK_{A \rightarrow B}$, then runs the re-encryption algorithm ReKeyGen to convert the original ciphertext C to transformed ciphertext C' and sends it to adversary $\mathcal{A}_1$.
- **Re-Decryption queries (ID_A, ID_B, C'):** When $\mathcal{A}_1$ asks for the decryption result of the transformed ciphertext C' , the challenger $\mathbb{C}$ runs the ReDecrypt algorithm and returns the decryption result to the adversary $\mathcal{A}_1$.

Challenge. When adversary $\mathcal{A}_1$ ends queries in Phase 1, it selects two messages $m_0, m_1 \in \{0, 1\}^{n_1}$ of the same length, and challenger $\mathbb{C}$ randomly selects $b \in \{0, 1\}$, computes the challenge ciphertext $C_b^* = \text{Encrypt}(param, m_b, PK_{ID^*})$ for the message m_b , and sends C_b^* to adversary $\mathcal{A}_1$.

Phase 2. The adversary $\mathcal{A}_1$ may continue with the Phase 1 queries with the following limitations.

- In both Phase 1 and Phase 2, the adversary cannot query the private key of the challenger identity ID^* and the trapdoor of the equality test.
- The original ciphertext of the ID^* cannot be obtained as a re-encryption ciphertext in Phase 1.
- Cannot query the decryption result of (ID^*, C_b^*) .

Guess. When the adversary $\mathcal{A}_1$ ends Phase 2, $\mathcal{A}_1$ outputs the guess bit $b' \in \{0, 1\}$. If $b' = b$, the adversary $\mathcal{A}_1$ wins. The winning advantage is defined as

$$\text{Adv}_{\mathcal{A}_1}^{\text{IND-sID-CCA}}(k) = \left| \Pr[b' = b] - \frac{1}{2} \right|.$$

Definition 1. If the advantage of adversary $\mathcal{A}_1$ in game 1 is negligible in k , the scheme PF-PREET is said to satisfy indistinguishability under selective identity and adaptive selective ciphertext attacks, noted as IND-sID-CCA security.

Security model 2. For type II adversary, the one-wayness under selective identity and adaptive selective ciphertext attacks is defined by means of Game 2, referred to as OW-sID-CCA security.

Setup. Same as in Game 1.

Phase 1. Same as in Game 1.

Challenge. When adversary $\mathcal{A}_2$ ends the query in Phase 1, the challenger $\mathbb{C}$ randomly selects a message $m^* \in \{0, 1\}^{n_1}$ and sends the ciphertext $C^* = \text{Encrypt}(param, m^*, PK_{ID^*})$ to adversary $\mathcal{A}_2$.

Phase 2. The adversary $\mathcal{A}_2$ may continue with the Phase 1 query with the following limitations.

- The private key of the challenge user ID^* cannot be queried, but the equality test key of the challenge user can be queried.
- The re-encryption query the original ciphertext of the ID^* cannot be queried.
- The decryption result of (ID^*, C_b^*) cannot be queried.

Guess. When adversary $\mathcal{A}_2$ ends Phase 2, $\mathcal{A}_2$ outputs a guess message $m' \in \{0, 1\}^{n_1}$. If $m' = m^*$, then adversary $\mathcal{A}_2$ wins. The winning advantage is defined as

$$\text{Adv}_{\mathcal{A}_2}^{\text{OW-sID-CCA}}(k) = \Pr[m' = m^*].$$

Definition 2. If the advantage of $\mathcal{A}_2$ in Game 2 is negligible in k , the scheme PF-PREET is said to satisfy one-wayness under the selective identity and adaptive selection ciphertext attacks, noted as OW-sID-CCA security.

3. Concrete construction

3.1. Scheme design

Setup(k). Enter a security parameter k to generate a cyclic group (G, q, g) , where q is the order of G and g is the generator of G . Assume that the message space is $\{0, 1\}^{n_1}$ and the length of elements in Z_q^* is n_2 bits. Choose secure hash functions $H_1 : \{0, 1\}^* \rightarrow Z_q^*$, $H_2 : \{0, 1\}^* \times \{0, 1\}^* \times Z_q^* \rightarrow Z_q^*$, $H_3 : Z_q^* \rightarrow \{0, 1\}^{n_1+n_2}$, $H_4 : \{0, 1\}^* \rightarrow Z_q^*$, $H_5 : Z_q^* \times Z_q^* \times \{0, 1\}^* \rightarrow \{0, 1\}^{2n_2}$. Randomly select $x \in Z_q^*$, set $msk = x$, compute the master public key $mpk = g^x$. KGC securely keeps msk , and publishes the system parameter

$$param = \{G, q, g, Z_q, mpk, H_1, H_2, H_3, H_4, H_5\}.$$

Extract($param, ID, msk$). Generate the key pairs of data owner and data user respectively. Input system parameter $param$, user identity $ID \in \{0, 1\}^*$ and master private key msk , randomly select $s_1, s_2 \in Z_q^*$, compute user's private key $SK = (SK_1, SK_2) = (s_1, s_2 + xH_1(ID))$ and user's public key $PK = (PK_1, PK_2) = (g^{s_1}, g^{s_2})$, and return (PK, SK) to the user.

Encrypt($param, m, PK$). This algorithm is used to encrypt a plaintext message to a ciphertext. Input system parameter $param$, plaintext message $m \in \{0, 1\}^{n_1}$, user's public key PK , randomly select $r_1, r_2 \in Z_q^*$ and compute ciphertext $C = (C_1, C_2, C_3, C_4)$.

$$\begin{aligned} k &= PK_2 \cdot mpk^{H_1(ID)} = g^{s_2+xH_1(ID)} \\ C_1 &= g^{r_1}, C_2 = g^{r_2} \\ C_3 &= (m \parallel r_2) \oplus H_3(k^{r_1}) \\ C_4 &= (H_4(m) \parallel r_2) \oplus H_5(g^{s_1 \cdot r_1}, C_2, C_3) \end{aligned} \quad (1)$$

C_3 is used to recover the data and C_4 is used to equality test. When the computation is completed, the data owner and data user send their respective ciphertext to the cloud server.

Decrypt($param, C, SK$). User uses this algorithm to decrypt ciphertext C . Input system parameter $param$, ciphertext C , and user's private key SK , it computes

$$m \parallel r_2 = C_3 \oplus H_3((C_1)^{sk_2}) \quad h_4 \parallel r_2' = C_4 \oplus H_3((C_1)^{sk_1}, C_2, C_3) \quad (2)$$

If $H_4(m) \neq h_4$ or $r_2 \neq r_2'$, it returns $\perp$. Otherwise, it uses the random number r_2 obtained from the decryption to verify $C_2 = g^{r_2}$. If the verification passes, it returns the decryption result m , otherwise it returns $\perp$.

Aut(SK). This algorithm is used to generate the authorization trapdoor for the equality test. Input the user's private key SK , it generates the trapdoor $td = SK_1$.

Test(C_A, td_A, C_B, td_B). This algorithm is used to determine whether the ciphertexts of two users contain the same messages. Input ciphertext $C_A = (C_{1,A}, C_{2,A}, C_{3,A}, C_{4,A})$ and $C_B = (C_{1,B}, C_{2,B}, C_{3,B}, C_{4,B})$, trapdoor $td_A = SK_{1,A}$ and $td_B = SK_{1,B}$, it computes

$$H_4(m_A) \parallel r_{2,A} = C_{4,A} \oplus H_5(C_{1,A}^{td_A}, C_{2,A}, C_{3,A}) \quad (3)$$

$$H_4(m_B) \parallel r_{2,B} = C_{4,B} \oplus H_5(C_{1,B}^{td_B}, C_{2,B}, C_{3,B}) \quad (4)$$

If $H_4(m_A) = H_4(m_B)$, it indicates that $m_A = m_B$; Otherwise, $m_A \neq m_B$.

ReKeyGen($param, ID_A, ID_B, SK_A, PK_B$). This algorithm is used to generate the re-encryption key from ID_A to ID_B . With the input of the system parameter $param$, the identity information of the two parties

ID_A and ID_B , the private key of the data owner SK_A , and the public key of the data user PK_B , the KGC first computes

$$\begin{aligned} R &= H_2 \left(ID_A \parallel ID_B \parallel (PK_{2,B} mpk^{H_1(ID_B)})^{SK_{1,A}} \right) \\ &= H_2 \left(ID_A \parallel ID_B \parallel (g^{s_{2,B}} g^{xH_1(ID_B)})^{s_{1,A}} \right) \end{aligned} \quad (5)$$

and then computes the re-encryption key $RK_{A \rightarrow B}$ using R ,

$$RK_{A \rightarrow B} = SK_{2,A} \cdot R^{-1} = (s_{2,A} + xH_1(ID_A)) R^{-1} \quad (6)$$

The re-encryption key $RK_{A \rightarrow B}$ is sent to the proxy server for re-encryption. In order to prevent the user private key from being leaked, the proxy server is required not to conspire with the storage server or user B . That is, the proxy server cannot send $RK_{A \rightarrow B}$ to the storage server or user B , and the storage server or user B cannot send R to the proxy server.

ReEncrypt($C, RK_{A \rightarrow B}$). This algorithm is used to generate the transformed ciphertext. Input the original ciphertext C and the re-encryption key $RK_{A \rightarrow B}$, it computes the transformed ciphertext $C' = (C_1', C_2', C_3')$, and then generates the transformed ciphertext

$$\begin{aligned} C_1' &= (C_1)^{RK_{A \rightarrow B}} = g^{r_{1,A} \cdot (s_{2,A} + xH_1(ID_A)) R^{-1}} \\ C_2' &= C_2 \\ C_3' &= C_3 \end{aligned} \quad (7)$$

After the encryption, it sends the transformed ciphertext to the data user B .

ReDecrypt($ID_A, ID_B, C', SK_B, PK_A$). This algorithm is run by the data user B and is used to obtain the plaintext of the transformed ciphertext. With the input of two party identities ID_A and ID_B , the transformed ciphertext $C' = (C_1', C_2', C_3')$, the data owner's public key PK_A and the data user's private key SK_B , it computes

$$\begin{aligned} R' &= H_2 \left(ID_A \parallel ID_B \parallel PK_{1,A}^{SK_{2,B}} \right) \\ &= H_2 \left(ID_A \parallel ID_B \parallel (g^{s_{2,B}} g^{xH_1(ID_B)})^{s_{1,A}} \right) \\ T &= (C_1')^{R'} = (g^{r_{1,A} \cdot (s_{2,A} + xH_1(ID_A)) R^{-1}})^{R'} \end{aligned} \quad (9)$$

Then, it uses R' and T to compute $m \parallel r_2 = C_3' \oplus H_3(T)$. Finally, it verifies the correctness of the decryption by using the random number r_2 to check $C_2' = g^{r_2}$. If the verification passes, it returns m ; Otherwise, it returns $\perp$.

Two notes on the above scheme.

Note 1. In the **Encrypt** algorithm, to avoid the server or others from impersonating the user to generate the data ciphertext, the identity-based digital signature technique (Li et al., 2019) can be introduced. By attaching a signature associated with the user's identity to the ciphertext, it can authenticate the source of the ciphertext, and thus avoids others from impersonating the user's identity to generate the data ciphertext.

Note 2. In the **ReKeyGen** algorithm, to avoid data recovery, it requires the data owner to generate the proxy re-encryption key online in real-time. The algorithm uses the trusted center KGC to generate the corresponding proxy re-encryption key, which requires KGC to store the data owner's private key and increases the storage burden of KGC to a certain extent. In fact, each user's key is generated by the KGC using the master private key and the user's identity. Among the keys of each user, s_1 and s_2 are randomly chosen by KGC. To avoid storage burden to store each user's private key, the KGC can generate these two random numbers using a pseudo-random function and based on the user's identity. When KGC needs to compute the proxy re-encryption key using the user's private key, it only needs to recompute these two random numbers using the pseudo-random function and the user's identity to recover the user's private key. Therefore, KGC only needs to save a pseudo-random function key K . Moreover, under the random oracle model, we can construct an efficient pseudo-random function $PRF(k, x) = H(k, x)$ using a hash function, where $k \in Z_q^*$ is the key

of the pseudo-random function, x is the input arbitrary bit string, and $H : Z_q^* \times \{0, 1\}^* \rightarrow Z_q^*$ is a hash function. When KGC wants to generate a key with the user identity ID , KGC first uses the pseudo-random function to compute

$$s_1 = PRF(k, ID \parallel 1) \text{ and } s_2 = PRF(k, ID \parallel 2) \quad (10)$$

then computes the user private key $SK = (SK_1, SK_2) = (s_1, s_2 + xH_1(ID))$ and the user's public key $PK = (PK_1, PK_2) = (g^{s_1}, g^{s_2})$.

3.1.1. Correctness of the original ciphertext decryption

For the original ciphertext $C = (C_1, C_2, C_3, C_4)$, the user can use the his private key to compute

$$\begin{aligned} C_1^{SK_2} &= C_1^{s_2 + xH_1(ID)} \\ &= (g^{r_1})^{s_2 + xH_1(ID)} \\ &= (g^{s_2 + xH_1(ID)})^{r_1} \\ &= k^{r_1} \end{aligned} \quad (11)$$

Decryption of the original ciphertext using $C_1^{SK_2}$,

$$\begin{aligned} m \parallel r_2 &= C_3 \oplus H_3(C_1^{SK_2}) \\ &= (m \parallel r_2) \oplus H_3(k^{r_1}) \oplus H_3(C_1^{s_2}) \\ &= (m \parallel r_2) \oplus H_3(k^{r_1}) \oplus H_3(k^{r_1}) \\ &= m \parallel r_2 \end{aligned} \quad (12)$$

To verify the decryption result, the user computes $\alpha = g^{r_2}$, if $\alpha = C_2$, then the user can correctly decrypt the original ciphertext and obtain the plaintext message m .

3.1.2. Correctness of transformed ciphertext decryption

For the transformed ciphertext $C' = (C'_1, C'_2, C'_3)$, using the public key of the data owner A and the private key of the data user B , it can compute

$$\begin{aligned} R' &= H_2(ID_A \parallel ID_B \parallel PK_{1,A}^{SK_{2,B}}) \\ &= H_2(ID_A \parallel ID_B \parallel (g^{s_{2,B} + xH_1(ID_B)})^{s_{1,A}}) \\ &= H_2(ID_A \parallel ID_B \parallel (PK_{2,B} \cdot mpk^{H_1(ID_B)})^{s_{1,A}}) \\ &= R \end{aligned} \quad (13)$$

and use R' to compute

$$\begin{aligned} T &= (C'_1)^{R'} = (C'_1)^{RK_{A \rightarrow B} \cdot R'} \\ &= g^{r_{1,A} \cdot (s_{2,B} + xH_1(ID_B)) \cdot R^{-1} \cdot R'} \\ &= g^{r_{1,A} \cdot (s_{2,B} + xH_1(ID_A))} \\ &= k^{r_{1,A}} \end{aligned} \quad (14)$$

and use T to compute

$$\begin{aligned} m \parallel r_2 &= C_3 \oplus H_3(T) \\ &= (m \parallel r_2) \oplus H_3(k^{r_1}) \oplus H_3(T) \\ &= (m \parallel r_2) \oplus H_3(k^{r_1}) \oplus H_3(k^{r_1}) \\ &= m \parallel r_2 \end{aligned} \quad (15)$$

To verify the decryption result, it computes $\beta = g^{r_2}$, if $\beta = C'_2$, then the user can correctly decrypt the transformed ciphertext to get the plaintext message m .

3.1.3. Correctness of the equality test

It uses the trapdoor to obtain the hash value of the plaintext message. Since the hash function is strongly collision-resistant, the same input will produce the same output, so if $H_4(m_A) = H_4(m_B)$, then

$m_A = m_B$ is determined.

$$\begin{aligned} &H_4(m_A) \parallel r_{2,A} \\ &= C_{4,A} \oplus H_5(C_{1,A}^{ID_A}, C_{2,A}, C_{3,A}) \\ &= H_5(g^{s_{1,A}r_{1,A}}, C_{2,A}, C_{3,A}) \oplus (H_4(m_A) \parallel r_{2,A}) \oplus H_5(C_{1,A}^{ID_A}, C_{2,A}, C_{3,A}) \\ &= H_5(g^{s_{1,A}r_{1,A}}, C_{2,A}, C_{3,A}) \oplus (H_4(m_A) \parallel r_{2,A}) \oplus H_5(g^{s_{1,A}r_{1,A}}, C_{2,A}, C_{3,A}) \\ &= H_4(m_A) \parallel r_{2,A} \end{aligned} \quad (16)$$

$$\begin{aligned} &H_4(m_B) \parallel r_{2,B} \\ &= C_{4,B} \oplus H_5(C_{1,B}^{ID_B}, C_{2,B}, C_{3,B}) \\ &= H_5(g^{s_{1,B}r_{1,B}}, C_{2,B}, C_{3,B}) \oplus (H_4(m_B) \parallel r_{2,B}) \oplus H_5(C_{1,B}^{ID_B}, C_{2,B}, C_{3,B}) \\ &= H_5(g^{s_{1,B}r_{1,B}}, C_{2,B}, C_{3,B}) \oplus (H_4(m_B) \parallel r_{2,B}) \oplus H_5(g^{s_{1,B}r_{1,B}}, C_{2,B}, C_{3,B}) \\ &= H_4(m_B) \parallel r_{2,B} \end{aligned} \quad (17)$$

The analysis of the correctness of the above three aspects concludes that this scheme is correct.

4. Security analysis

This scheme is based on the Gap-CDH problem for security analysis. The Gap-CDH problem is constructed by the CDH problem and the DDH problem. We first introduce the CDH and the DDH problems, and finally lead to the Gap-CDH problem.

CDH Problem. Suppose G is a cyclic group of order q , q is a prime, and g is a generator of G . Then the CDH problem is described as follows: randomly take two elements a, b from Z_q^* , given the tuple (g, g^a, g^b) , to compute g^{ab} . Let $\mathcal{A}$ be a Probabilistic Polynomial Time (PPT) adversary. Its advantage to break the CDH problem is defined as

$$\text{Adv}_{\mathcal{A}}^{\text{CDH}} = \Pr[\mathcal{A}(g, g^a, g^b) \rightarrow g^{ab}] \leq \epsilon \quad (18)$$

If ϵ is negligible, then $\mathcal{A}$ is said to be difficult to solve the CDH problem.

DDH problem. Suppose G is a cyclic group of order q , q is a prime, and g is a generator of G . Then the DDH problem is described as follows: randomly take three elements a, b, c from Z_q^* , and given the tuple (g, g^a, g^b, g^c) , determine whether g^{ab} is equal to g^c . Let $\mathcal{A}$ be the PPT adversary. Its advantage to break the DDH problem is defined as

$$\text{Adv}_{\mathcal{A}}^{\text{DDH}} = \left| \Pr[\mathcal{A}(g, g^a, g^b, g^{ab})] - \Pr[\mathcal{A}(g, g^a, g^b, g^c)] \right| \leq \epsilon \quad (19)$$

If ϵ is negligible, then $\mathcal{A}$ is said to be difficult to solve the DDH problem.

Gap-CDH problem. Based on the CDH problem and the DDH problem, let $\mathcal{A}$ be the PPT adversary that can solve the DDH problem. Consider the experiment $\text{Experiment}_{\mathcal{A}}^{\text{Gap-CDH}}$: randomly take two elements a, b from Z_q^* , given the tuple (g, g^a, g^b) and a DDH problem random oracle $\mathcal{O}(U, V)$, which returns True only when $V = U^a$, and False otherwise. If the PPT adversary $\mathcal{A}$ can output g^{ab} , then it returns True.

The advantage of being able to solve the Gap-CDH problem in polynomial time is

$$\text{Adv}_{\mathcal{A}}^{\text{Gap-CDH}} = \Pr[\text{Experiment}_{\mathcal{A}}^{\text{Gap-CDH}} = 1] \leq \epsilon \quad (20)$$

If ϵ is negligible, then it is said that $\mathcal{A}$ is difficult to solve the Gap-CDH problem.

The following security analysis is performed using the above Gap-CDH problem.

Theorem 1. If the Gap-CDH problem is difficult, then the scheme PF-PREET satisfies IND-sID-CCA security against the type I adversary in the random oracle model.

Proof. Let $\mathcal{A}_1$ be a type I adversary attacking the security of the PF-PREET scheme IND-sID-CCA. Construct an algorithm Sim below to solve the Gap-CDH problem with the help of $\mathcal{A}_1$. Suppose $(G, q, g, g^a, g^b, O(\cdot, \cdot))$ is an instance of the Gap-CDH problem, where the random oracle $O(U, V)$ returns True if and only if $V = U^a$; Otherwise, it returns False. Assume that the challenge user chosen by the adversary is ID^* . Sim simulates the execution of Game 1 as follows.

Setup. Sim implicitly sets the system master private key as $msk = a$ and makes the master public key as $mpk = g^a$.

Next, Sim selects the hash functions H_1, H_2, H_3, H_4, H_5 in the same way as the original scheme and sends the system parameters to the adversary $\mathcal{A}_1$. In addition, Sim randomly selects $s_1', s_2' \in \mathbb{Z}_q^*$, and computes the public key of the challenging user $PK_{ID^*} = (PK_1^*, PK_2^*) = (g^{a+s_1'}, g^{s_2'+a \cdot H_1(ID^*)})$ and returns this public key to the adversary. Here, the private key of the challenging user is implicitly defined as $SK_{ID^*} = (SK_1^*, SK_2^*) = (a + s_1', s_2' + a \cdot H_1(ID^*))$.

phase 1. When $\mathcal{A}_1$ makes the following queries, Sim responds in the following manner.

- **Hash query.** For each hash function H_i , Sim creates a list $L_i = \{\langle I_{i,j}, O_{i,j} \rangle\}$, where $I_{i,j}$ represents the input of the hash function and $O_{i,j}$ represents the output result. If the hash value of $I_{i,j}$ has been asked in the previous hash query, the corresponding hash value is output. Otherwise, a value $O_{i,j}$ is randomly selected as the corresponding hash value and stored in the list L_i , thus conforming to the nature of random oracle.
- **Key query (ID).** When $\mathcal{A}_1$ asks for the key of ID , if $ID = ID^*$, Sim refuses to answer the query. Otherwise, Sim randomly selects the user's private key $SK_{ID} = (SK_1, SK_2) \in \mathbb{Z}_q^* \times \mathbb{Z}_q^*$, and computes the user's public key $PK_1 = g^{SK_1}$ and $PK_2 = \frac{g^{SK_2}}{g^{aH_1(ID)}}$. Finally, Sim sends the private key SK_{ID} and the public key PK_{ID} to the adversary $\mathcal{A}_1$. In addition, Sim saves the key pairs of the user.
- **Decryption query (ID, C).** Assume the ciphertext is $C = (C_1, C_2, C_3, C_4)$. If $ID \neq ID^*$, Sim can use the private key of ID to decrypt directly and return the result to the adversary $\mathcal{A}_1$. If $ID = ID^*$, Sim recovers $m \parallel r_2$ from the ciphertext C_3 in the following way.
 - Sim creates a list with elements of the form $L_{3,j}' = \{\langle g^{r_{3,j}}, O_{3,j}' \rangle\}$.
 - For each element $g^{r_{3,j}}$ in the list $L_{3,j}'$, if there exists $g^{r_{3,j}} = C_1$, Sim computes $m \parallel r_2 = C_3 \oplus O_{3,j}'$. Otherwise, Sim iterates over each element $I_{3,j}$ in the list $L_3 = \{\langle I_{3,j}, O_{3,j} \rangle\}$ and computes $V = \left(\frac{I_{3,j}}{C_1 s_2'} \right)^{H_1(ID)^{-1}}$, if $O(C_1, V) = 1$, Sim computes $m \parallel r_2 = C_3 \oplus O_{3,j}$. Otherwise, Sim randomly chooses $m \parallel r_2 \in \{0, 1\}^{n_1+n_2}$.

Similarly, Sim recovers $H_4(m) \parallel r_2$ from C_4 , and then verifies the correctness of the ciphertext. If the verification passes, the message m is returned. Otherwise, the symbol $\perp$ is returned, indicating that the ciphertext form is not legitimate.

- **Trapdoor query (ID_i).** When $\mathcal{A}_1$ queries ID_i for a trapdoor, since Sim saves the private key of ID_i (except for challenging users), Sim can run the algorithm $\text{Aut}(SK_{ID_i})$ to compute the equality test trapdoor td_{ID_i} , and return it to $\mathcal{A}_1$.
- **Re-encryption query (ID_A, ID_B).** Since this security model does not allow the attacker to query the challenging user for ciphertext re-encryption, when $\mathcal{A}_1$ queries ID_A to ID_B about the transformed ciphertext C , Sim has the private key of ID_A and can generate the re-encryption key $RK_{A \rightarrow B}$ by the algorithm ReKeyGen and then run the re-encryption algorithm ReEncrypt to convert the original ciphertext C to transformed ciphertext C' and send it to the adversary $\mathcal{A}_1$.

- **Re-decryption query (ID_A, ID_B, C').** When $\mathcal{A}_1$ asks for the decryption result of the transformed ciphertext C' , if $ID_B \neq ID^*$, Sim can directly run the re-decryption algorithm ReDecrypt and return the decryption result to the adversary $\mathcal{A}_1$ since Sim has the private key of ID_B . If $ID_B = ID^*$, Sim can first compute R' using the private key of ID_A , and then compute T . Finally, decrypting transformed ciphertext to get result and return it to $\mathcal{A}_1$.

Challenge. When the adversary $\mathcal{A}_1$ ends the query of Phase 1, the adversary $\mathcal{A}_1$ selects two plaintext messages of the same length $m_0, m_1 \in \{0, 1\}^{n_1}$ and sends them to Sim. Sim randomly selects $b \in \{0, 1\}$ and $r_2 \in \mathbb{Z}_q^*$ and computes the challenge ciphertext $C_b^* = (C_1^*, C_2^*, C_3^*, C_4^*)$ for the plaintext message m_b , where $C_1^* = g^b$, $C_2^* = g^{r_2}$, $C_3^* \leftarrow \{0, 1\}^{n_1+n_2}$ and $C_4^* \leftarrow \{0, 1\}^{2n_2}$, then sends it to the adversary.

Phase 2. Sim answers the adversary's queries in the same way as in phase 1. The difference is that when the adversary makes a decryption query, if the ciphertext $C_1 = C_1^*$ is in $C = (C_1, C_2, C_3, C_4)$, Sim returns the symbol $\perp$ directly. Otherwise, Sim decrypts the answer in the same way as in phase 1.

Guess. Finally, $\mathcal{A}_1$ outputs guessing bits $b' \in \{0, 1\}$. If $b' = b$, then the adversary $\mathcal{A}_1$ wins.

Analysis. In the Gap-CDH problem, since both a and b are randomly selected, the master private key and system parameters of the above simulated game are selected in the same way as the actual game. The distribution of the public and private keys of the challenge user is the same as the user keys in the original scheme, and the distribution of the elements C_1^* and C_2^* in the challenge ciphertext is also the same as the distribution of the actual ciphertext. In all types of queries, except for the decryption query of the challenge user ciphertext, the other queries are also answered in the same way as the actual model. For the decryption queries in phase 1, Sim is able to correctly determine whether the inputs of the hash functions H_3 and H_5 in the decryption algorithm appear in the existing list of hash queries by using the DDH judgment oracle random. If it appears, Sim performs the decryption query according to the result in the hash list. If it does not appear, Sim randomly selects the hash value and performs the decryption. This is consistent with the way the hash function is computed in the random oracle model. Therefore, the decryption query in phase 1 is consistent with the original model. For the decryption query in phase 2, when $C_1 = C_1^*$ and $C \neq C^*$, in the original model, the challenger $\mathcal{C}$ will answer according to the decryption algorithm, while in the above simulation game, Sim directly returns the symbol $\perp$, indicating that the form of the ciphertext is not legitimate. The following discussion is divided into two cases.

If $C_3 \neq C_3^*$, unless the adversary asks for the H_3 hash of $(k^*)^a$ in the challenge ciphertext, the decrypted random number r_2 is completely random and cannot pass the verification of the ciphertext legitimacy. If the adversary asks for the H_3 hash of $(k^*)^a$ in the challenge ciphertext, Sim can directly use $(k^*)^a$ to solve the example of Gap-CDH problem.

If $C_3 = C_3^*$, then the message m and the random number r_2 must be the same as in the challenge ciphertext, thus the ciphertext $C = C^*$. This case is not allowed for decryption query.

The above analysis shows that the game process simulated by Sim is consistent with the game process in the original security model.

Let event E indicate that the adversary has asked H_3 hash of $g^{(s_2'+a \cdot H_1(ID^*))b}$ or H_5 hash of $(g^{(a+s_1')b}, *, *)$ in the hash function query, where “*” denotes any element. Obviously, if E never occurs, then C_3^* and C_4^* in the challenge ciphertext are completely random, and the probability of $b' = b$ is $1/2$. It is proved below that if the adversary has the advantage of attacking the scheme IND-sID-CCA security in the simulation game as ϵ_1 , then the probability that the event E occurs is at least $2\epsilon_1$. Since

$$\begin{aligned} \Pr[b'=b] &= \Pr[b' = b \wedge E] + \Pr[b' = b \wedge \neg E] \\ &\leq \Pr[E] + \Pr[b' = b | \neg E] \cdot \Pr[\neg E] \\ &= \frac{1}{2} + \frac{1}{2} \cdot \Pr[E] \end{aligned} \quad (21)$$

$$\begin{aligned}
\Pr[b'=b] &= \Pr[b' = b \wedge E] + \Pr[b' = b \wedge \neg E] \\
&\geq \Pr[b' = b | \neg E] \cdot \Pr[\neg E] \\
&= \frac{1}{2} - \frac{1}{2} \cdot \Pr[E]
\end{aligned} \tag{22}$$

it follows that $\left| \Pr[b' = b] - \frac{1}{2} \right| \leq \frac{1}{2} \Pr[E]$, that is $\Pr[E] \geq 2\epsilon_1$.

If the event E occurs, Sim can find $g^{(s_2' + a \cdot H_1(ID^*)) \cdot b}$ or $g^{(a + s_1') \cdot b}$ from the adversary's hash query. Since Sim knows s_1' and s_2' , and can solve the Gap-CDH problem g^{ab} from them. This completes the proof.

Theorem 2. *If the Gap-CDH problem is difficult, then the scheme PF-PREET satisfies OW-sID-CCA security against type II adversary in the random oracle model.*

Proof. Type II adversary can query the equality test key of the challenging user, except that the attack capability is the same as that of Type I adversary. Therefore, the proof procedure of Theorem 2 is basically the same as that of Theorem 1, with the difference that it includes.

- When generating the challenge user's key, Sim randomly selects s_1^* as the first part of the challenge user's private key SK_1^* and computes the public key $PK_1^* = g^{s_1^*}$, while the second part of the private key SK_2^* and the public key PK_2^* are selected in the same way as in Theorem 1.
- When the adversary asks the user (including the challenging user) for the equality test key, Sim returns the first part of the user's private key SK_1 to the adversary.
- When performing a decryption query to challenging user, Sim uses the private key SK_1^* to recover $H_4(m) \parallel r_2$ directly from C_4 .
- In the challenge phase, Sim randomly selects a message $m^* \in \{0, 1\}^{n_1}$ and a random number $r_2 \in Z_q^*$ from the message space. It computes the challenge ciphertext $C^* = (C_1^*, C_2^*, C_3^*, C_4^*)$ and sends it to the adversary, where $C_1^* = g^b$, $C_2^* = g^{r_2}$, $C_3^* \leftarrow \{0, 1\}^{n_1+n_2}$ and $C_4^* = (H_4(m^*) \parallel r_2) \oplus H_5(g^{r_1 \cdot s_1^*}, C_2, C_3)$.
- In the guess, the adversary outputs the guess message $m' \in \{0, 1\}^{n_1}$.

Analysis. Make the event F indicate that the adversary has asked for the H_3 hash of $g^{(s_2' + a \cdot H_1(ID^*)) \cdot b}$. If the event F never occurs, the adversary does not get any information about the challenge message m^* from the ciphertext C_3^* , but only the H_4 hash of the message from C_4^* . Assuming that the adversary asks for the hash of H_4 at most q_{H_4} , the probability that the adversary can obtain the challenge message at most $\frac{1}{2^{n_2 - q_{H_4}}}$ under the random oracle model. It is proved below that if the adversary has the advantage of attacking the security of the scheme OW-sID-CCA in the simulation game as ϵ_2 , then the probability of the event F occurring is at least $\frac{N+1}{N} \epsilon_2 - \frac{1}{N}$, where $N = 2^{n_2} - q_{H_2} - 1$. Since

$$\begin{aligned}
\Pr[m' = m^*] &= \Pr[m' = m^* \wedge F] + \Pr[m' = m^* \wedge \neg F] \\
&\leq \Pr[F] + \Pr[m' = m^* | \neg F] \cdot (1 - \Pr[F]) \\
&= \frac{1}{2^{n_2} - q_{H_2}} + \left(1 - \frac{1}{2^{n_2} - q_{H_2}}\right) \cdot \Pr[F]
\end{aligned} \tag{23}$$

So, $\Pr[F] \geq \frac{N+1}{N} \epsilon_2 - \frac{1}{N}$.

If the event F occurs, similar to Theorem 1, Sim can obtain the element $g^{(s_2' + a \cdot H_1(ID^*)) \cdot b}$ from the hash list of H_3 and then solve the Gap-CDH problem.

5. Performance analysis

5.1. Theoretical analysis

This section focuses on theoretical analyzing the computational overhead of this paper's scheme in terms of encryption, decryption,

and authorization, and comparing the scheme with previous proxy re-encryption schemes with equality test (Yang et al., 2021; Li et al., 2022; Chen et al., 2021; Hadabi et al., 2024), and some pairing-free proxy re-encryption schemes without equality test (Lu and Li, 2016; Prasad and Purushothama, 2021; Yan et al., 2024). The comparison of security and computational overhead is carried out.

Li et al. (2022) proposed a scheme that satisfies one-wayness under the selective ciphertext attack for adversary that cannot access the user's private key, denoted as OW-CCA security, and indistinguishability under the selective ciphertext attack for adversary that can access the user's private key, denoted as IND-CCA security. Yang et al. (2021) proposed a new scheme to improve Li et al.'s scheme by verifying the integrity of the ciphertext during decryption to achieve IND-CCA security for both original and transformed ciphertexts. Chen et al. (2021) proposed that IND-CCA security is satisfied for an external adversary who can access to the trapdoor. Recently, Hadabi et al. (2024) proposed an IND-CCA secure PRE scheme with plaintext checkable encryption.

Using Alice and Bob to denote the two parties involved in the test, the authorization levels included in the chosen schemes are as follows.

Level I denotes user-level authorization. The tester can test any ciphertext of Alice and Bob when tester has the authorization of Alice and Bob.

Level II denotes the ciphertext level authorization. The tester can perform the equality test on the ciphertexts when the tester has the authorization for the specified ciphertexts of Alice and Bob.

Level III denotes that the user specifies the level of authorization. Alice specifies the user who performs the equality test, and the tester can only perform the equality test on all ciphertexts of Alice with those of the specified user.

The difficult issues involved in the chosen solution include the following. The DCDH denotes the Divisible Computation Diffie-Hellman problem. The DBDH denotes the Decisional Bilinear Diffie-Hellman problem.

First, some relevant attributes of the scheme are listed in Table 1 to compare the security, difficult problems and authorization levels. The authorization level is used to indicate the range of the equality test, where authorization level I indicates the largest test range, and authorization levels II and III test ranges are gradually reduced. It can be seen that the scheme in this paper can reach the authorization test of level I, while other schemes can only reach the authorization test of level II or level III with the same security, or without providing equality test function.

Among the existing PRE schemes with equality test, ours scheme is partially identity-based, Hadabi et al.'s scheme is identity-based, while the other schemes are non-identity-based. Depending on the security model, the non-identity-based PRE schemes with equality test may achieve either IND-CCA or OW-CCA, where IND-CCA allows an adversary to adaptively select ciphertexts for attack while guaranteeing the semantic security of the scheme. However, OW-CCA also allows adversary to adaptively select ciphertexts for attacks, but the scheme can only achieve one-wayness. The security model of this scheme allows an adversary to adaptively select ciphertexts for attack, while semantic security or one-wayness can be achieved. In addition, the scheme's model allows the adversary to select the user's identity for the attack, and any of the IND-sID-CCA secure or OW-sID-CCA secure identity-based encryption schemes can be transformed into a traditional (non-identity-based) IND-CCA secure or OW-CCA secure scheme (Boneh et al., 2007).

Table 2 analyzes the computational overhead. In the table, 'E' denotes exponential operation, 'P' denotes bilinear pairing operation, 'H' denotes map-to-point hash operation. We ignore the other operations, such as general hash, modular multiplication and modular inversion, as they are negligible. '-' denotes that the scheme does not contain the corresponding algorithm. In the encryption algorithm, compared with the paper (Chen et al., 2021), our scheme adds one exponential operation, which increases the computational overhead. In the

Table 1

The comparison of security, assumptions and authorization.

Scheme	Security	Assumptions	Authorization
Yang et al. (2021)	IND-CCA	DCDH	II
Li et al. (2022)	IND-CCA,OW-CCA	DCDH	II
Chen et al. (2021)	IND-CCA	CDH,DBDH	III
Hadabi et al. (2024)	IND-ID-CCA	DBDH	II
Lu and Li (2016)	IND-CCA	CHD	–
Prasad and Purushothama (2021)	IND-CCA	DDH	–
Yan et al. (2024)	IND-CCA	CDH	–
Ours	IND-sID-CCA,OW-sID-CCA	Gap-CDH	I

Table 2

Comparison of computational overhead.

Schemes	Yang et al. (2021)	Li et al. (2022)	Chen et al. (2021)	Hadabi et al. (2024)	Lu and Li (2016)	Prasad and Purushothama (2021)	Yan et al. (2024)	Ours
Encrypt	7E+H	7E+1P+H	4E	E+P	4E	3E	5E	5E
Decrypt	E+2P	E	E	E+5P	2E	3E	4E	2E
Aut	E	E	3E	–	–	–	–	0
Test	2P	4P+2H	2P	3P	–	–	–	2E
ReKeyGen	2E	2E	2E	0	2E	0	2E	2E
ReEncrypt	E	2E	0	3P	3E	5E	5E	E
ReDecrypt	4E+2P+H	4E	5E	4P	4E	4E	4E	2E

decryption algorithm, to ensure CCA security, our scheme verifies the correctness of decryption and adds one exponential operation, which increases the computational overhead compared with the paper (Li et al., 2022; Chen et al., 2021). But, it has lower computation complexity than that of Yang et al. (2021) and Hadabi et al. (2024). In the authorization algorithm, depending on the authorization level, its computational overhead increases accordingly. The authorization mode of our scheme is user-level authorization, and its test range is relatively more extensive. Under the same security, the extended authorization scope compared with the paper (Yang et al., 2021; Li et al., 2022; Chen et al., 2021; Hadabi et al., 2024) is more in line with the application scenario of secure de-duplication of massive IoT node data and can save computational overhead. Compared with the pairing-free PRE schemes (Lu and Li, 2016; Prasad and Purushothama, 2021; Yan et al., 2024), our scheme supports both encrypted data sharing and de-duplication, while the others only supports encrypted data sharing.

5.2. Experimental simulation

This section further evaluates our scheme and the schemes (Yang et al., 2021; Li et al., 2022; Chen et al., 2021). The performance of the scheme is simulated using the following computer equipment environment: AMD Ryzen7 5700U-1.80 GHz processor, 8 GB RAM, and Windows 10 operating system. In order to provide proper security, we selected Type-A bilinear pairing. This pairing is constructed based on the elliptic curve $y^2 = x^3 + x \bmod p$, where $p \equiv 3 \bmod 4$. Type-A bilinear pairing generates a symmetric group, so the order of the cyclic group G_1, G_2, G_T is 512 bit and the Modulo of the integer group Z_q^* is 160 bit.

The main workflow of PREET scheme is divided into equality test and proxy re-encryption. So we compute the time consumption of equality test, proxy re-encryption and overall. The equality test time contains the running time of algorithms Encrypt, Decrypt, Aut and Test. While the proxy re-encryption time contains the running time of algorithms ReKeyGen, ReEncrypt and ReDecrypt. The average value, taking 100 times to obtain the overall time consumption of our scheme, is about 87.89 ms. While paper (Yang et al., 2021) is about 151.86 ms, paper (Li et al., 2022) is about 149.29 ms, paper (Chen et al., 2021) is about 111.08 ms and paper (Hadabi et al., 2024) is about 149.81 ms. It can be seen from Fig. 2 that the computational overhead of our scheme is better than that of the currently known PREET schemes in all phases and overall. Compared with the pairing-free PRE schemes (Lu and Li, 2016; Prasad and Purushothama, 2021; Yan et al., 2024), the total time of our scheme is till lower than that of them.

Fig. 3 shows a comparison of the running time of the algorithms in each phase of the selected schemes and our scheme. It can be seen that the computation time consumed in the encryption phase and decryption phase of our scheme is slightly higher than that of the selected PREET schemes, but our scheme outperforms the rest of the PREET schemes in terms of Aut, Test, and ReDecrypt. Especially, the ReDec algorithm is faster than all the other schemes. Especially, the ReDec algorithm of our scheme is the fastest one among all schemes. The IoT cloud platform provides data storage services for a large number of nodes, so the data security de-duplication process requires multiple equality test. A single equality test in our scheme is about 12.56 ms, which is only 1/3 of paper (Li et al., 2022).

In order to simulate the IoT working environment with a large amount of data and test the performance of the above scheme in the test process, 1000, 2000, 3000, 4000 and 5000 test labels are stored in the database beforehand. The size of each file is about 1 MB. The experimental results are shown in Fig. 4, which shows that when the computational overhead of the Test algorithm is larger in the case of a large amount of data for equality test, it will have a greater impact on the overall. Our scheme does not use bilinear pairing when constructing the Test algorithm, and the computational overhead is lower than that of papers (Yang et al., 2021; Li et al., 2022; Chen et al., 2021). As the PRE schemes (Lu and Li, 2016; Prasad and Purushothama, 2021; Yan et al., 2024) did not support equality test, we do not simulate the running time of Test algorithms of these schemes. Therefore, in the actual IoT environment, the scheme of this paper has certain advantages.

In summary, through theoretical analysis and experimental simulation, the proposed scheme is better than the currently proposed PREET schemes, even than those pairing-free PRE schemes without equality test in general. But, it still has some shortcomings, e.g., the running time of encryption algorithm and decryption algorithms is not well. Especially, in encryption algorithm, it requires more exponential operations for achieving IND-CCA security while maintaining data de-duplication. To improve efficiency of encryption algorithm, we may use a message authentication codes instead of the re-computation of g^{r^2} to check the validity of a ciphertext.

6. Conclusion

This paper combined PKEET and PRE schemes to construct a pairing-free proxy re-encryption equality test scheme PF-PREET, and proved its security using the Gap-CDH problem in the random oracle

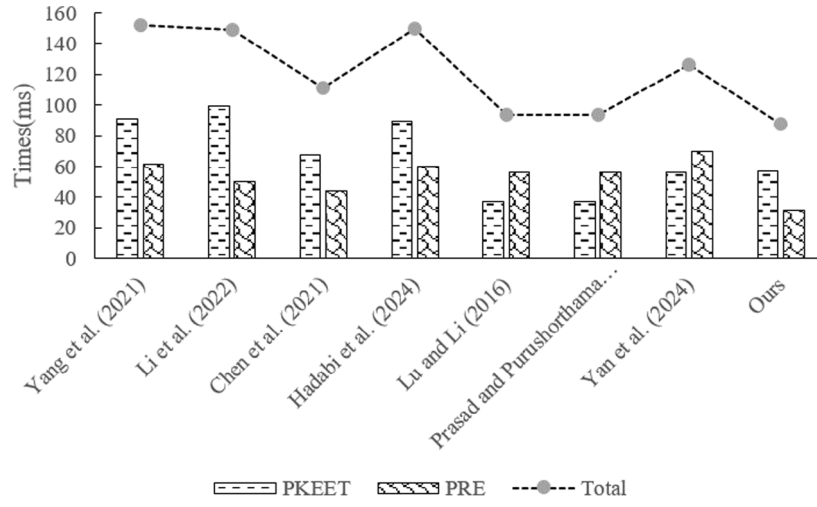


Fig. 2. Overall running time of schemes.

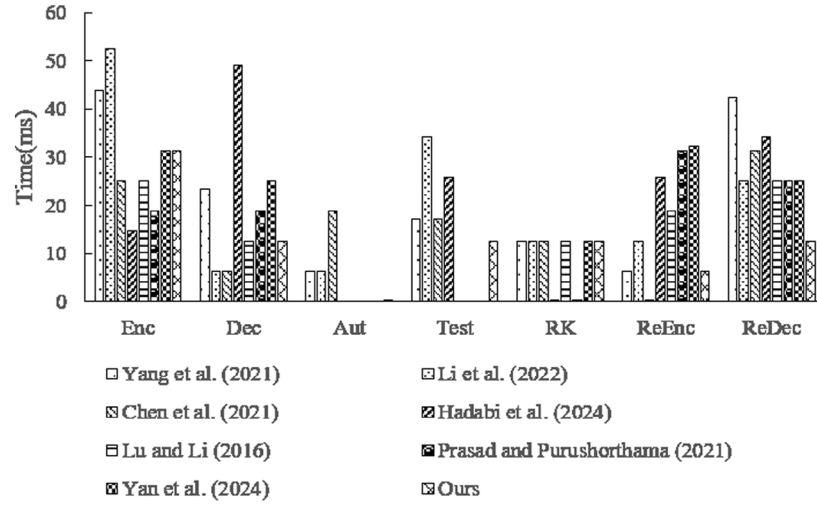


Fig. 3. Average running time of each phase of schemes.

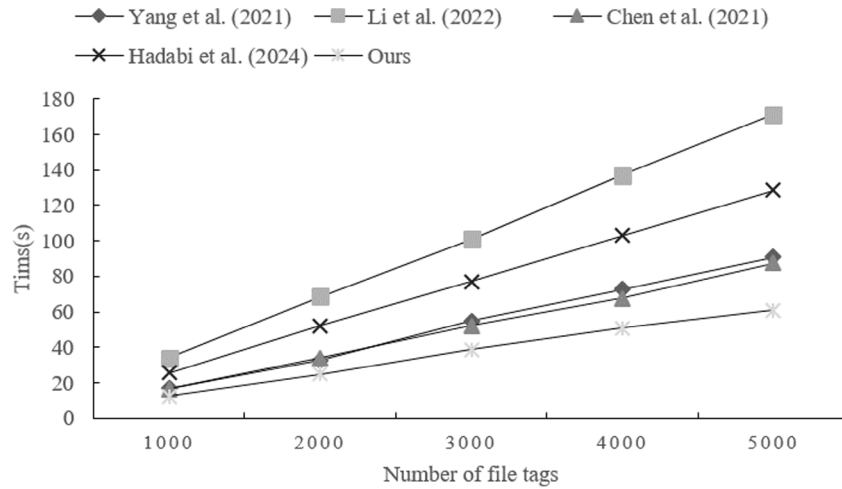


Fig. 4. Time overhead of tag test.

model. The scheme was designed without bilinear mapping operations, which improved the computational efficiency compared with existing PREET schemes through theoretical analysis and experimental

simulation, in terms of overall running time, average running time of each phase and time overhead of tag test. As for the secure deduplication of massive data in IoT environment, an efficient security

de-duplication algorithm can effectively improve the system efficiency. PKEET can realize the ciphertext equality test, and it combines with proxy re-encryption to realize a complete data secure de-duplication and recovery.

CRedit authorship contribution statement

Gang Han: Writing – review & editing, Supervision, Funding acquisition, Conceptualization. **Le Li:** Writing – original draft, Software. **Baodong Qin:** Writing – review & editing, Supervision, Methodology. **Dong Zheng:** Writing – review & editing, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by the National Natural Science Foundation of China (Grant Nos. 62102312, 62072371) and by the fund of State Key Laboratory of Integrated Services Networks (Grant No. ISN24-13)

References

- Agyekum, K.O.O., Xia, Q., Sifah, E.B., Cobblah, C.N.A., Xia, H., Gao, J., 2022. A proxy re-encryption approach to secure data sharing in the internet of things based on blockchain. *IEEE Syst. J.* 16 (1), 1685–1696. <http://dx.doi.org/10.1109/JSYST.2021.3076759>.
- Boneh, D., Canetti, R., Halevi, S., Katz, J., 2007. Chosen-ciphertext security from identity-based encryption. *SIAM J. Comput.* 36 (5), 1301–1328. <http://dx.doi.org/10.1137/S009753970544713X>.
- Boneh, D., Di Crescenzo, G., Ostrovsky, R., Persiano, G., 2004. Public key encryption with keyword search. In: Cachin, C., Camenisch, J. (Eds.), *Advances in Cryptology - EUROCRYPT 2004*, International Conference on the Theory and Applications of Cryptographic Techniques, Interlaken, Switzerland, May 2–6, 2004, Proceedings. In: *Lecture Notes in Computer Science*, Vol. 3027, Springer, pp. 506–522. http://dx.doi.org/10.1007/978-3-540-24676-3_30.
- Braeken, A., Shabisha, P., Touhafi, A., Steenhaut, K., 2017. Pairing free and implicit certificate based signcryption scheme with proxy re-encryption for secure cloud data storage. In: 2017 3rd International Conference of Cloud Computing Technologies and Applications (CloudTech). IEEE, pp. 1–7.
- Canto, A.C., Kaur, J., Kermani, M.M., Azarderakhsh, R., 2023a. Algorithmic security is insufficient: A comprehensive survey on implementation attacks haunting post-quantum security. <http://dx.doi.org/10.48550/ARXIV.2305.13544>, CoRR abs/2305.13544 arXiv:2305.13544.
- Canto, A.C., Kaur, J., Kermani, M.M., Azarderakhsh, R., 2023b. ChatGPT vs. Lightweight security: First work implementing the NIST cryptographic standard ASCON. <http://dx.doi.org/10.48550/ARXIV.2306.08178>, CoRR abs/2306.08178 arXiv:2306.08178.
- Canto, A.C., Kermani, M.M., Azarderakhsh, R., 2023c. Reliable architectures for finite field multipliers using cyclic codes on FPGA utilized in classic and post-quantum cryptography. *IEEE Trans. Very Large Scale Integr. Syst.* 31 (1), 157–161. <http://dx.doi.org/10.1109/TVLSI.2022.3224357>.
- Canto, A.C., Kermani, M.M., Azarderakhsh, R., Gaj, K., 2022. CRC-Oriented error detection architectures of post-quantum cryptography niederreiter key generator on FPGA. In: Nurmi, J., Wisland, D.T., Aunet, S., Kjølgaard, K. (Eds.), *IEEE Nordic Circuits and Systems Conference, NorCAS 2022*, Oslo, Norway, October 25–26, 2022. IEEE, pp. 1–7. <http://dx.doi.org/10.1109/NORCAS57515.2022.9934378>.
- Chen, B., He, D., Kumar, N., Wang, H., Choo, K.R., 2021. A blockchain-based proxy re-encryption with equality test for vehicular communication systems. *IEEE Trans. Netw. Sci. Eng.* 8 (3), 2048–2059. <http://dx.doi.org/10.1109/TNSE.2020.2999551>.
- Elhabob, R., Zhao, Y., Hassan, A., Xiong, H., 2020. PKE-ET-HS: Public key encryption with equality test for heterogeneous systems in IoT. *Wirel. Pers. Commun.* 113 (1), 313–335. <http://dx.doi.org/10.1007/S11277-020-07190-9>.
- Elhabob, R., Zhao, Y., Sella, I., Xiong, H., 2019. Public key encryption with equality test for heterogeneous systems in cloud computing. *KSII Trans. Internet Inf. Syst.* 13 (9), 4742–4770. <http://dx.doi.org/10.3837/TIIS.2019.09.023>.
- Elkhatib, R., Azarderakhsh, R., Kermani, M.M., 2021. Accelerated RISC-V for SIKE. In: 28th IEEE Symposium on Computer Arithmetic, ARITH 2021, Lyngby, Denmark, June 14–16, 2021. IEEE, pp. 131–138. <http://dx.doi.org/10.1109/ARITH51176.2021.00035>.
- Fan, X., Liu, F., 2019. Proxy re-encryption and re-signatures from lattices. In: Deng, R.H., Gauthier-Umaña, V., Ochoa, M., Yung, M. (Eds.), *Applied Cryptography and Network Security - 17th International Conference, ACNS 2019*, Bogota, Colombia, June 5–7, 2019, Proceedings. In: *Lecture Notes in Computer Science*, Vol. 11464, Springer, pp. 363–382. http://dx.doi.org/10.1007/978-3-030-21568-2_18.
- Hadabi, A., Qu, Z., Elhabob, R., Kumar, S., Yeh, K.-H., Kumari, S., Xiong, H., 2024. Proxy re-encryption with plaintext checkable encryption for integrating digital twins into IIoT. *Comput. Electr. Eng.* (ISSN: 0045-7906) 116, 109164. <http://dx.doi.org/10.1016/j.compeleceng.2024.109164>, URL <https://www.sciencedirect.com/science/article/pii/S0045790624000922>.
- Hassan, A., Elhabob, R., Ibrahim, U., Wang, Y., 2020. Efficient public key cryptography scheme with equality test for heterogeneous systems in IIoT. In: Wang, G., Chen, B., Li, W., Pietro, R.D., Yan, X., Han, H. (Eds.), *Security, Privacy, and Anonymity in Computation, Communication, and Storage - SpaCCS 2020 International Workshops*, Nanjing, China, December 18–20, 2020, Proceedings. In: *Lecture Notes in Computer Science*, Vol. 12383, Springer, pp. 108–121. http://dx.doi.org/10.1007/978-3-030-68884-4_9.
- Hong, H., Sun, Z., 2018. Sharing your privileges securely: a key-insulated attribute based proxy re-encryption scheme for IoT. *World Wide Web* 21 (3), 595–607. <http://dx.doi.org/10.1007/S11280-017-0475-8>.
- Huang, K., Tso, R., Chen, Y., Rahman, S.M.M., Almogren, A., Alamri, A., 2015. PKE-AET: Public key encryption with authorized equality test. *Comput. J.* 58 (10), 2686–2697. <http://dx.doi.org/10.1093/COMJNL/BXV025>.
- Kaur, J., Canto, A.C., Kermani, M.M., Azarderakhsh, R., 2023. A comprehensive survey on the implementations, attacks, and countermeasures of the current NIST lightweight cryptography standard. <http://dx.doi.org/10.48550/ARXIV.2304.06222>, CoRR abs/2304.06222 arXiv:2304.06222.
- Kermani, M.M., Azarderakhsh, R., 2016. Lightweight hardware architectures for fault diagnosis schemes of efficiently-maskable cryptographic substitution boxes. In: 2016 IEEE International Conference on Electronics, Circuits and Systems, ICECS 2016, Monte Carlo, Monaco, December 11–14, 2016. IEEE, pp. 764–767. <http://dx.doi.org/10.1109/ICECS.2016.7841314>.
- Kermani, M.M., Azarderakhsh, R., Ren, K., Beuchat, J., 2016. Guest Editorial: Introduction to the special section on emerging security trends for biomedical computations, devices, and infrastructures. *IEEE ACM Trans. Comput. Biol. Bioinform.* 13 (3), 399–400. <http://dx.doi.org/10.1109/TCBB.2016.2518874>.
- Khashan, O.A., 2020. Hybrid lightweight proxy re-encryption scheme for secure fog-to-things environment. *IEEE Access* 8, 66878–66887. <http://dx.doi.org/10.1109/ACCESS.2020.2984317>.
- Kim, S., Lee, I., 2018. IoT device security based on proxy re-encryption. *J. Ambient Intell. Humaniz. Comput.* 9 (4), 1267–1273. <http://dx.doi.org/10.1007/S12652-017-0602-5>.
- Lee, H.T., Ling, S., Seo, J.H., Wang, H., 2016. Semi-generic construction of public key encryption and identity-based encryption with equality test. *Inform. Sci.* 373, 419–440. <http://dx.doi.org/10.1016/J.IINS.2016.09.013>.
- Li, H., Huang, Q., Ma, S., Shen, J., Susilo, W., 2019. Authorized equality test on identity-based ciphertexts for secret data sharing via cloud storage. *IEEE Access* 7, 25409–25421. <http://dx.doi.org/10.1109/ACCESS.2019.2899680>.
- Li, W., Jin, C., Kumari, S., Xiong, H., Kumar, S., 2022. Proxy re-encryption with equality test for secure data sharing in Internet of Things-based healthcare systems. *Trans. Emerg. Telecommun. Technol.* 33 (10), <http://dx.doi.org/10.1002/ETT.3986>.
- Lin, H., Gao, F., Zhang, H., Jin, Z., Li, W., Wen, Q., 2022. Public key encryption with equality test supporting flexible designated authorization in cloud storage. *IEEE Syst. J.* 16 (1), 1460–1470. <http://dx.doi.org/10.1109/JSYST.2021.3106701>.
- Lu, Y., Li, J., 2016. A pairing-free certificate-based proxy re-encryption scheme for secure data sharing in public clouds. *Future Gener. Comput. Syst.* 62, 140–147. <http://dx.doi.org/10.1016/J.FUTURE.2015.11.012>.
- Ma, S., 2016. Identity-based encryption with outsourced equality test in cloud computing. *Inform. Sci.* 328, 389–402. <http://dx.doi.org/10.1016/J.IINS.2015.08.053>.
- Ma, S., Huang, Q., Zhang, M., Yang, B., 2015a. Efficient public key encryption with equality test supporting flexible authorization. *IEEE Trans. Inf. Forensics Secur.* 10 (3), 458–470. <http://dx.doi.org/10.1109/TIFS.2014.2378592>.
- Ma, S., Zhang, M., Huang, Q., Yang, B., 2015b. Public key encryption with delegated equality test in a multi-user setting. *Comput. J.* 58 (4), 986–1002. <http://dx.doi.org/10.1093/COMJNL/BXU026>.
- Muhammad, M.H.G., Alyas, T., Ahmad, F., Butt, F.H., Qazi, W.M., Saqib, S., 2021. An analysis of security challenges and their perspective solutions for cloud computing and IoT. *EAI Endorsed Trans. Scalable Inf. Syst.* 8 (30), e7. <http://dx.doi.org/10.4108/EAI.23-10-2020.166718>.
- Myers, S.A., Shull, A., 2018. Practical revocation and key rotation. In: Smart, N.P. (Ed.), *Topics in Cryptology - CT-RSA 2018 - the Cryptographers' Track At the RSA Conference 2018*, San Francisco, CA, USA, April 16–20, 2018, Proceedings. In: *Lecture Notes in Computer Science*, Vol. 10808, Springer, pp. 157–178. http://dx.doi.org/10.1007/978-3-319-76953-0_9.
- Prasad, S., Purushothama, B.R., 2021. CCA secure and efficient proxy re-encryption scheme without bilinear pairing. *J. Inf. Secur. Appl.* 58, 102703. <http://dx.doi.org/10.1016/J.JISA.2020.102703>.
- Qu, H., Yan, Z., Lin, X.J., Zhang, Q., Sun, L., 2018. Certificateless public key encryption with equality test. *Inform. Sci.* 462, 76–92. <http://dx.doi.org/10.1016/J.IINS.2018.06.025>.

- Roy, P.S., Duong, D.H., Susilo, W., Sipasseuth, A., Fukushima, K., Kiyomoto, S., 2022. Lattice-based public-key encryption with equality test supporting flexible authorization in standard model. *Theoret. Comput. Sci.* 929, 124–139. <http://dx.doi.org/10.1016/J.TCS.2022.06.034>.
- Sarker, A., Kermani, M.M., Azarderakhsh, R., 2022. Efficient error detection architectures for postquantum signature Falcon's Sampler and KEM SABER. *IEEE Trans. Very Large Scale Integr. Syst.* 30 (6), 794–802. <http://dx.doi.org/10.1109/TVLSI.2022.3156479>.
- Susilo, W., Guo, F., Zhao, Z., Wu, G., 2022. PKE-MET: Public-key encryption with multi-ciphertext equality test in cloud computing. *IEEE Trans. Cloud Comput.* 10 (2), 1476–1488. <http://dx.doi.org/10.1109/TCC.2020.2990201>.
- Tang, Q., 2012. Public key encryption supporting plaintext equality test and user-specified authorization. *Secur. Commun. Netw.* 5 (12), 1351–1362. <http://dx.doi.org/10.1002/SEC.418>.
- Thirumalai, C., Mohan, S., Srivastava, G., 2020. An efficient public key secure scheme for cloud and IoT security. *Comput. Commun.* 150, 634–643. <http://dx.doi.org/10.1016/J.COMCOM.2019.12.015>.
- Wang, Z., 2018. Leakage resilient ID-based proxy re-encryption scheme for access control in fog computing. *Future Gener. Comput. Syst.* 87, 679–685. <http://dx.doi.org/10.1016/J.FUTURE.2017.12.001>.
- Wu, L., Zhang, Y., Choo, K.R., He, D., 2017. Efficient and secure identity-based encryption scheme with equality test in cloud computing. *Future Gener. Comput. Syst.* 73, 22–31. <http://dx.doi.org/10.1016/J.FUTURE.2017.03.007>.
- Yan, L., Qin, H., Yang, K., Xie, H., Wang, X.A., Liu, S., 2024. Pairing-free certificate-based proxy re-encryption plus scheme for secure cloud data sharing. *Electronics* 13 (3), <http://dx.doi.org/10.3390/electronics13030534>, URL <https://www.mdpi.com/2079-9292/13/3/534>.
- Yang, G., Tan, C.H., Huang, Q., Wong, D.S., 2010. Probabilistic public key encryption with equality test. In: Pieprzyk, J. (Ed.), *Topics in Cryptology - CT-RSA 2010, the Cryptographers' Track At the RSA Conference 2010, San Francisco, CA, USA, March 1-5, 2010. Proceedings*. In: *Lecture Notes in Computer Science*, Vol. 5985, Springer, pp. 119–131. http://dx.doi.org/10.1007/978-3-642-11925-5_9.
- Yang, C., Tso, R., Liu, Z., Hsu, J., Tseng, Y., 2021. Improved proxy re-encryption scheme with equality test. In: *16th Asia Joint Conference on Information Security, AsiaJCIS 2021, Seoul, Republic of Korea, August 19-20, 2021*. IEEE, pp. 37–44. <http://dx.doi.org/10.1109/ASIAJCIS53848.2021.00016>.
- Zuo, C., Shao, J., Liu, J.K., Wei, G., Ling, Y., 2018. Fine-grained two-factor protection mechanism for data sharing in cloud storage. *IEEE Trans. Inf. Forensics Secur.* 13 (1), 186–196. <http://dx.doi.org/10.1109/TIFS.2017.2746000>.